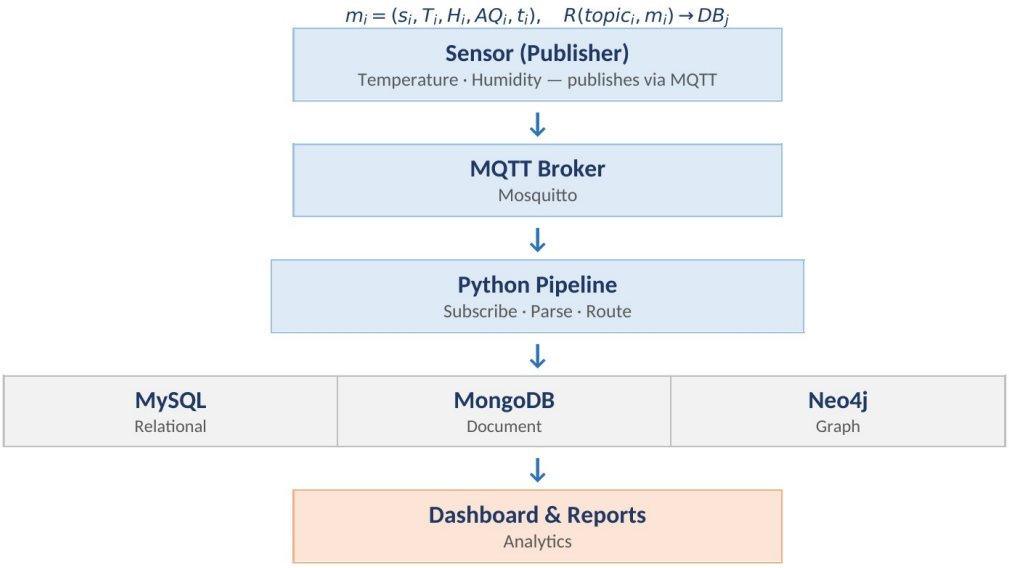
UNIVERSITÀ DEGLI STUDI DI MESSINA
Bachelor Degree in Data Analysis · Data Analysis Curriculum
Database Management Systems — Mod. B

PROJECT REPORT

IoT-Based Environmental Monitoring System

From Physical Sensors to Digital Dashboards and Multi-Database Data Pipelines



STUDENT
Adonay Samson Gebrewold

MATRICOLA
570352

ACADEMIC YEAR
2025 – 2026

PROFESSOR
Prof. Armando Ruggeri

GITHUB
Donnie2444

PROJECT
DB-B1

Contents

1. Introduction

The purpose of this project is to build an IoT-based environmental monitoring system that receives sensor data through MQTT, processes it with Python, and stores the result in different database platforms. The final implementation focuses on environmental monitoring, where simulated sensors produce temperature, humidity, and air quality values.

The project follows the idea of a real-time data pipeline. A sensor publisher sends messages to an MQTT broker, the Python subscriber receives those messages, and the subscriber decides where to store the data according to the MQTT topic. This is the central design decision of the project: different topics represent different kinds of information, and each kind of information is stored in the database model that fits it best.

The system uses MySQL for structured readings and alerts, MongoDB for raw telemetry with flexible device metadata, and Neo4j for the sensor/location topology. A Streamlit dashboard is also included to read from the three databases and present the stored data in one interface.

2. Project Motivation and Objective

The number of IoT devices used in buildings, laboratories, factories, and smart environments is increasing quickly. These devices continuously generate small but frequent data messages. If the messages are not organized correctly, it becomes difficult to analyze the data, detect problems, and store information efficiently.

Environmental monitoring is a practical example of this problem. A room may need to be monitored for high temperature, high humidity, or poor air quality. The values are simple, but they become useful only when they are received continuously, classified, stored, and made available for analysis.

The objective of this project is therefore to create an integrated system that can receive MQTT messages, process the data in Python, route each message by topic, and store the data in SQL, document, and graph databases. The project also evaluates whether the system works end to end through functional tests, database queries, dashboard views, and simple performance observations.

3. System Architecture

The final architecture is based on topic-based routing. The publisher does not send one generic message to all databases. Instead, it publishes different kinds of messages to different MQTT topics. The subscriber listens to all of these topics and writes each message to the correct database.

MQTT topic	Type of data	Target database	Reason
environmental/readings	Structured measurements: temperature, humidity, air quality, timestamp	MySQL	Fixed schema, SQL queries, status classification, and alerts
environmental/telemetry	Raw device telemetry and optional metadata	MongoDB	Flexible JSON documents where fields can vary per message
environmental/network	Sensor placement and topology	Neo4j	Graph representation of Sensor, Room, Floor, and Building relationships

The topic-based design can also be described formally. Each incoming MQTT message is treated as a tuple, and the routing function maps the message and its topic to the correct database model.

$$\mathcal{M} = \{m_i\}_{i=1}^{n}, \quad m_i = (s_i, T_i, H_i, AQ_i, t_i)$$
$$R(topic_i, m_i) \rightarrow DB_j, \quad DB_j \in \{\text{SQL}, \text{MongoDB}, \text{Neo4j}\}$$

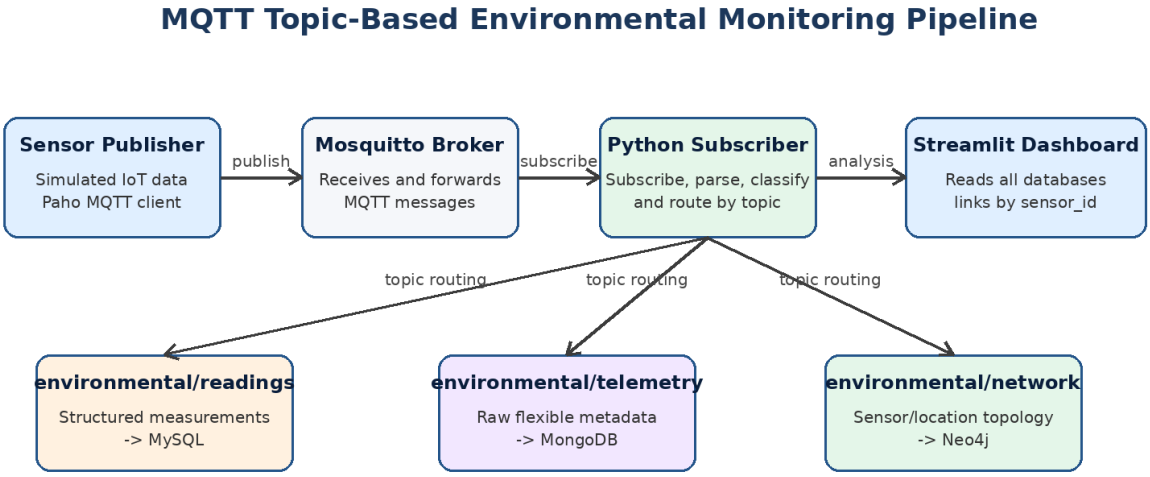


Figure 1: Final topic-based architecture of the environmental monitoring system.

The architecture is intentionally simple, but it demonstrates an important database principle: the same IoT system can contain several types of data, and one database model is not always the best solution for every type. This is why the project uses a relational database, a document database, and a graph database together.

4. Technologies Used

4.1 Python

Python is used to implement the two main application programs. The publisher simulates environmental sensors and generates data. The subscriber connects to the MQTT broker, receives messages, parses JSON payloads, performs the processing logic, and writes to the databases.

4.2 MQTT and Eclipse Mosquitto

MQTT is used as the communication protocol between the simulated IoT devices and the processing application. Eclipse Mosquitto was selected as the MQTT broker because it is lightweight, widely used for IoT projects, and easy to run inside Docker. The broker listens on port 1883 and forwards published messages to subscribed clients.

4.3 MySQL

MySQL is used for structured environmental readings and alerts. This part of the project benefits from tables, primary keys, relational constraints, and SQL queries. The readings table stores the measured values, while the alerts table stores warning or danger events linked to readings.

4.4 MongoDB

MongoDB is used to store raw telemetry messages. This is useful because telemetry may contain optional fields, such as signal strength, error codes, firmware version, or maintenance notes. These fields do not appear in every message, so a document database is a natural fit.

4.5 Neo4j

Neo4j is used to store the sensor/location network. In the final implementation, Neo4j is not used for every reading. It is used for the topology: sensors are located in rooms, rooms are on floors, and floors belong to a building. This allows the system to answer relationship questions and link sensor readings to physical locations.

4.6 Docker and Streamlit

Docker Compose is used to run Mosquitto, MySQL, MongoDB, and Neo4j as containers. Streamlit is used to create the dashboard. The dashboard is not the core database pipeline, but it makes the result easier to inspect and explain because it displays the three database models in one place.

5. MQTT Server Implementation

The MQTT server layer was implemented using Eclipse Mosquitto. Mosquitto acts as the broker between the publisher and the subscriber. The publisher sends messages to the broker, and the subscriber receives only the messages for the topics it subscribed to.

The subscriber subscribes to three topics when it connects to the broker:

environmental/network

environmental/readings

environmental/telemetry

This design is important because the MQTT topic is not only a communication label. In this project, the topic also carries meaning. It tells the subscriber which database should receive the message.

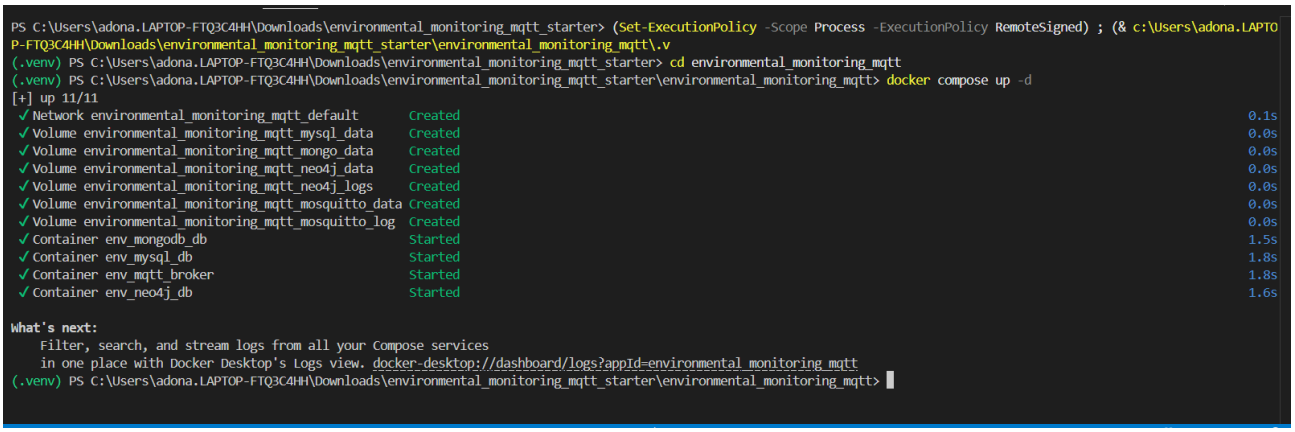


Figure 2: Docker Compose creates and starts the Mosquitto, MySQL, MongoDB, and Neo4j services.

6. Python Application Development

6.1 Publisher Program

The publisher simulates three environmental sensors: S001, S002, and S003. Each sensor belongs to a known room and floor. At the beginning of execution, the publisher sends the sensor topology to the network topic so Neo4j can build the location graph. Then it continuously generates environmental readings every two seconds.

For every cycle, the publisher sends two main messages: one reading message to MySQL and one telemetry message to MongoDB. The reading message contains the structured environmental values. The telemetry message contains the same device context plus flexible metadata such as battery level, firmware version, optional signal strength, optional error code, and optional maintenance note.

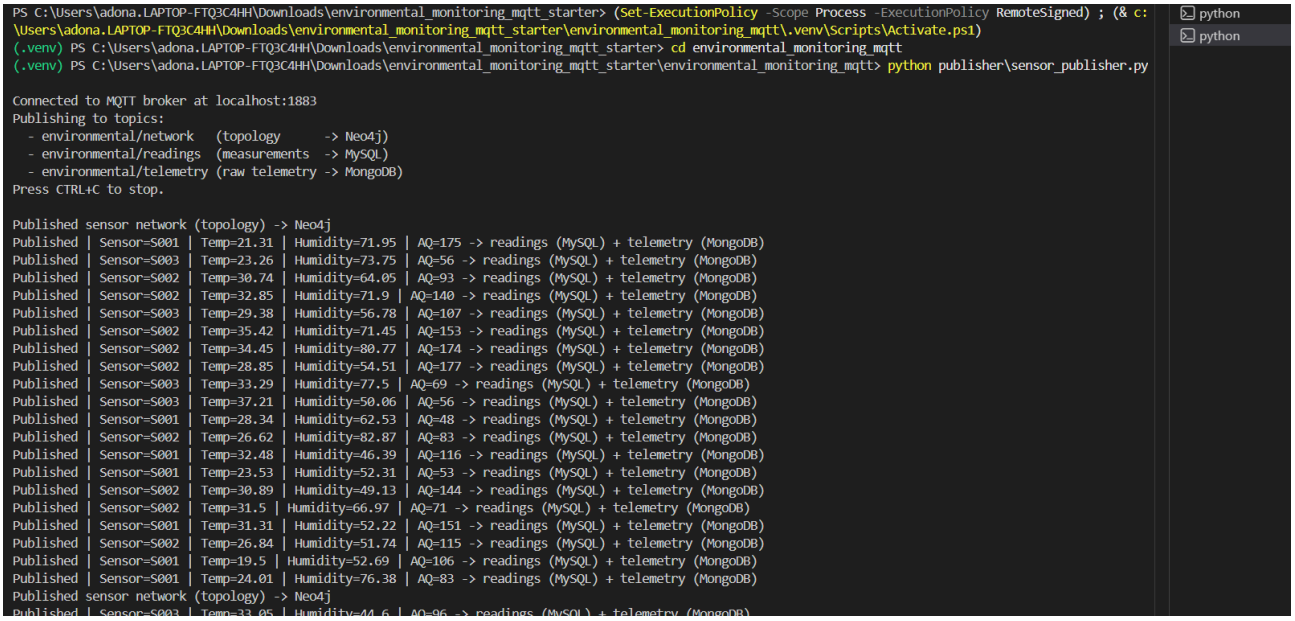


Figure 3: Publisher sending topic-based messages: network data to Neo4j, readings to MySQL, and telemetry to MongoDB.

6.2 Subscriber Program

The subscriber is the processing and routing part of the pipeline. It connects to the broker, subscribes to the three topics, receives messages, decodes JSON payloads, and then calls the correct database function according to the topic.

if topic == "environmental/readings":

write structured reading and alert to MySQL

elif topic == "environmental/telemetry":

write raw telemetry document to MongoDB

elif topic == "environmental/network":

write topology nodes and relationships to Neo4j

This routing logic is the heart of the project because it directly satisfies the requirement that data should be written to different database platforms based on the MQTT message topic.

Figure 4: Subscriber connected to Mosquitto and subscribed to the three final MQTT topics.

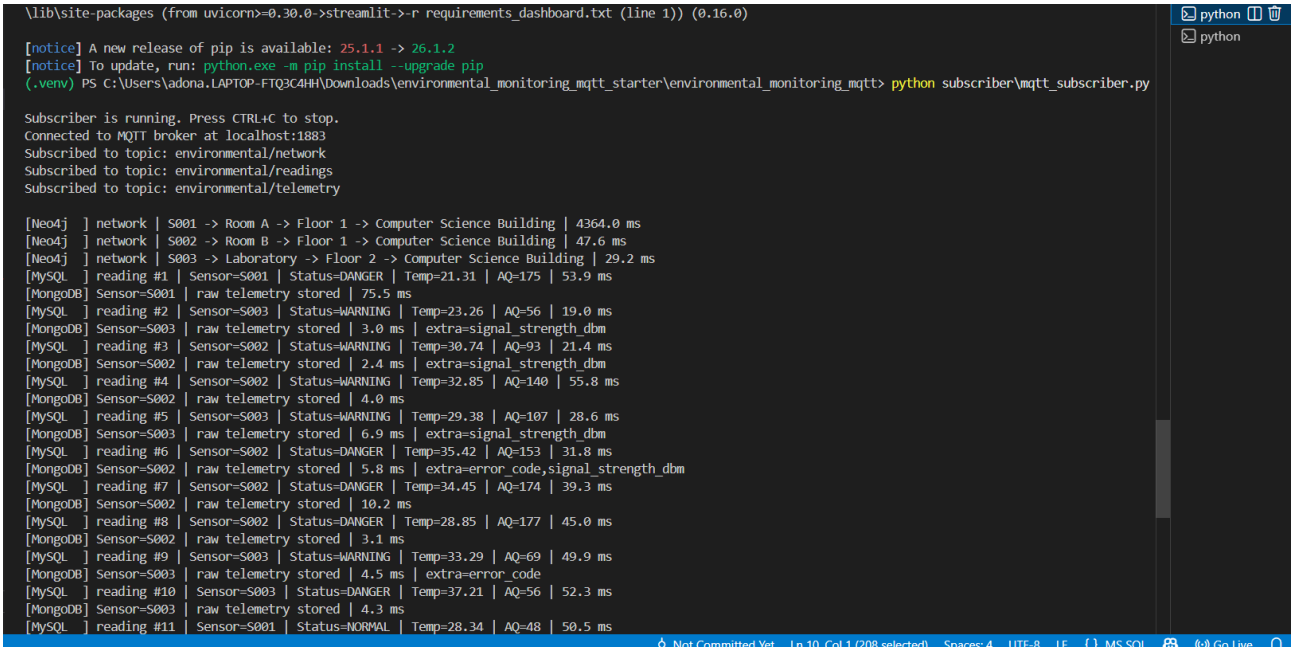


Figure 5: Subscriber receiving messages, classifying readings, and writing each topic to the correct database.

6.3 Data Processing Logic

The subscriber does more than simply save incoming data. For structured readings, it classifies each environmental condition into NORMAL, WARNING, or DANGER. The classification rules are based on thresholds for temperature, humidity, and air quality.

$$\text{DANGER}: T_i > 35 \ \vee \ H_i > 80 \ \vee \ AQ_i > 150$$
$$\text{WARNING}: T_i > 30 \ \vee \ H_i > 70 \ \vee \ AQ_i > 100$$
$$\text{NORMAL}: \text{otherwise}$$

Condition	Rule
DANGER	temperature > 35, or humidity > 80, or air_quality > 150
WARNING	temperature > 30, or humidity > 70, or air_quality > 100
NORMAL	none of the warning or danger conditions are true

When a warning or danger condition is detected, the subscriber inserts an alert record in MySQL. The alert stores an alert type, and the human-readable message is taken from the alert_types table. This avoids repeating the same alert text many times and keeps the relational design cleaner.

7. Database Integration and Design

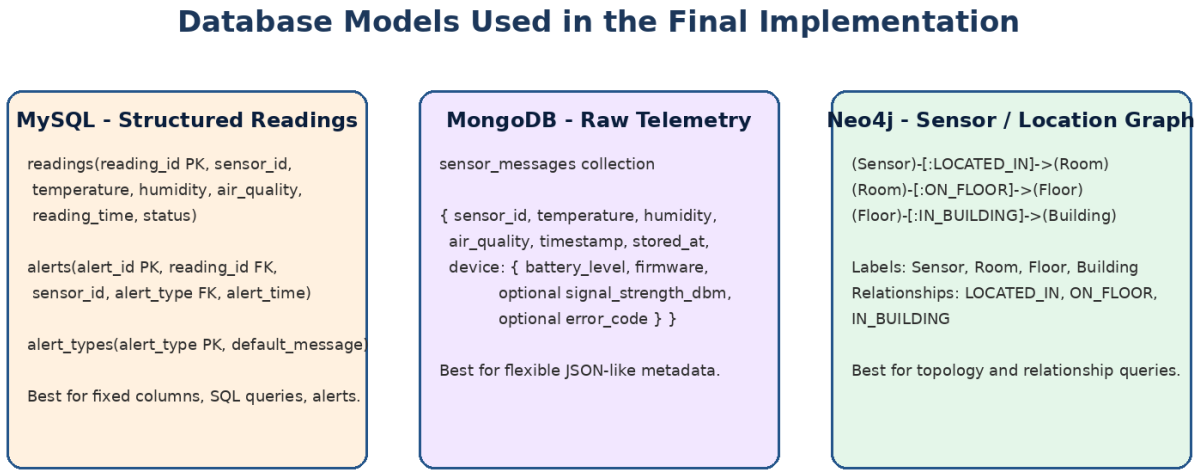


Figure 6: Final database models used by the project.

7.1 MySQL Relational Design

MySQL stores the structured and processed part of the system. The main table is readings, which contains the sensor ID, temperature, humidity, air quality, reading time, and status. When the status indicates a warning or danger condition, an alert is inserted into the alerts table.

The current MySQL design is centered on the following tables:

readings(reading_id PK, sensor_id, temperature, humidity, air_quality, reading_time, status, created_at)

alerts(alert_id PK, reading_id FK, sensor_id, alert_type FK, alert_time)

alert_types(alert_type PK, default_message)

This design is appropriate for SQL because readings and alerts have predictable columns and can be queried using GROUP BY, ORDER BY, JOIN, and aggregate functions.

7.2 MongoDB Document Design

MongoDB stores raw telemetry messages as documents. This part of the system keeps the device message close to the format in which it arrived. The document contains the sensor values, timestamp, storage time, and a nested device object.


```
{
 "sensor_id": "S001",
 "temperature": 21.31,
 "humidity": 71.95,
 "air_quality": 175,
 "timestamp": "2026-06-29 14:16:05",
 "device": {
  "battery_level": 57,
  "firmware_version": "v2.0.1",
  "signal_strength_dbm": -59
 },
 "stored_at": "2026-06-29 14:16:10"
```

}

The important point is that not every telemetry message has the same shape. Some messages contain signal strength, some contain an error code, and some may contain a maintenance note. This flexible structure is the reason MongoDB is used.

7.3 Neo4j Graph Design

Neo4j stores the location network instead of the measurement stream. The final graph contains four labels: Sensor, Room, Floor, and Building. The relationships describe how sensors are physically placed in the building.

(Sensor)-[:LOCATED_IN]->(Room)

(Room)-[:ON_FLOOR]->(Floor)

(Floor)-[:IN_BUILDING]->(Building)

This graph model is useful because the dashboard can map a MySQL reading to a physical room by using sensor_id. MySQL stores the reading itself, while Neo4j stores where the sensor is located.

7.4 Why Three Database Models Were Used

The project uses three database models because each one solves a different part of the problem. MySQL is strong for structured data and analysis queries. MongoDB is strong for raw messages with flexible metadata. Neo4j is strong for relationships and topology. This approach is called polyglot persistence, meaning that a system uses more than one storage model when the data requirements are different.

8. Dashboard and Analysis Layer

A Streamlit dashboard was developed to show the result of the data pipeline. The dashboard connects to MySQL, MongoDB, and Neo4j. It displays connection status, system totals, latest MySQL readings, charts for environmental values over time, raw MongoDB telemetry, and a Neo4j sensor/location summary.

The dashboard also demonstrates cross-database analysis. MySQL readings contain sensor_id, and Neo4j knows the room for each sensor. The dashboard joins these two pieces of information so that readings can be shown with their location.

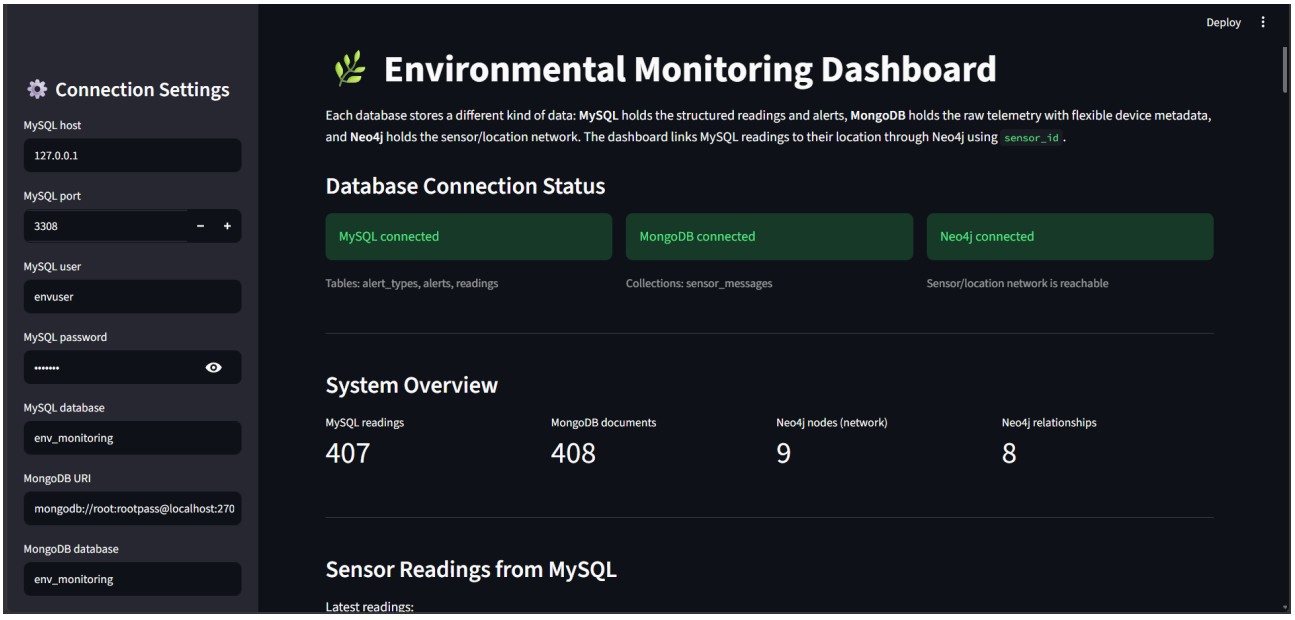


Figure 7: Streamlit dashboard showing successful database connections and the system overview.

Sensor Readings from MySQL

Latest readings:

	reading_id	sensor_id	temperature	humidity	air_quality	reading_time	status	created_at	location
406	407	S001	19.91	44.44	133	2026-06-29 14:29:39	WARNING	2026-06-29 12:29:39	Room A
405	406	S002	18.21	68.32	74	2026-06-29 14:29:37	NORMAL	2026-06-29 12:29:37	Room B
404	405	S003	31.84	54.15	134	2026-06-29 14:29:35	WARNING	2026-06-29 12:29:35	Laboratory
403	404	S001	30.07	63.25	57	2026-06-29 14:29:33	WARNING	2026-06-29 12:29:33	Room A
402	403	S002	25.27	72.23	76	2026-06-29 14:29:31	WARNING	2026-06-29 12:29:31	Room B
401	402	S002	24.52	59.52	53	2026-06-29 14:29:29	NORMAL	2026-06-29 12:29:29	Room B
400	401	S001	26.55	50.89	74	2026-06-29 14:29:27	NORMAL	2026-06-29 12:29:27	Room A
399	400	S002	24.34	45.82	134	2026-06-29 14:29:25	WARNING	2026-06-29 12:29:25	Room B
398	399	S003	30.55	50.86	140	2026-06-29 14:29:23	WARNING	2026-06-29 12:29:23	Laboratory
397	398	S002	33.92	36.84	59	2026-06-29 14:29:21	WARNING	2026-06-29 12:29:21	Room B

Figure 8: Dashboard view of latest MySQL readings with location joined from Neo4j.

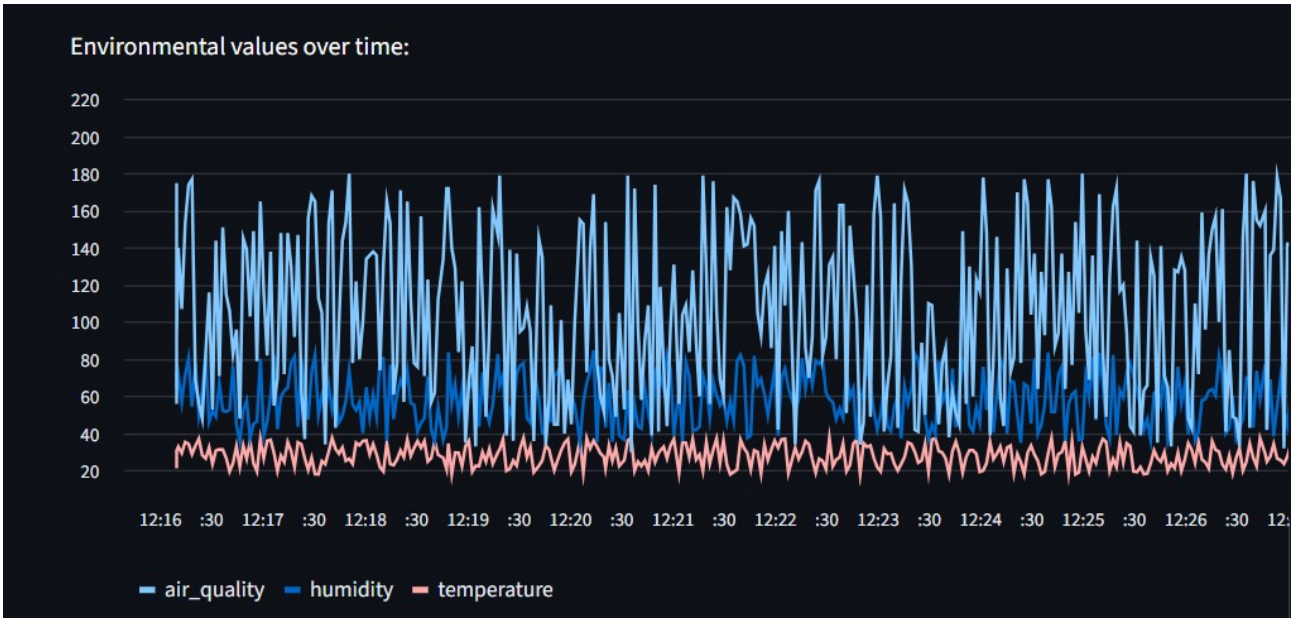


Figure 9: Dashboard chart showing temperature, humidity, and air quality over time.

Raw Telemetry from MongoDB

MongoDB stores the raw device messages, including flexible metadata that varies per message.

	_id	sensor_id	temperature	humidity	air_quality	timestamp	stored_at	device.battery_level	device.firmware_v
0	6a4265353f16420bb2661fc2	S003	19.79	70.97	118	2026-06-29 14:29:41	2026-06-29 14:29:41	93	v1.2.0
1	6a4265333f16420bb2661fc1	S001	19.91	44.44	133	2026-06-29 14:29:39	2026-06-29 14:29:39	24	v2.0.1
2	6a4265313f16420bb2661fc0	S002	18.21	68.32	74	2026-06-29 14:29:37	2026-06-29 14:29:37	62	v1.0.3
3	6a42652f3f16420bb2661fbf	S003	31.84	54.15	134	2026-06-29 14:29:35	2026-06-29 14:29:35	34	v1.0.3
4	6a42652d3f16420bb2661fbe	S001	30.07	63.25	57	2026-06-29 14:29:33	2026-06-29 14:29:33	41	v1.2.0
5	6a42652b3f16420bb2661fbd	S002	25.27	72.23	76	2026-06-29 14:29:31	2026-06-29 14:29:31	33	v1.2.0
6	6a4265293f16420bb2661fbc	S002	24.52	59.52	53	2026-06-29 14:29:29	2026-06-29 14:29:29	52	v2.0.1
7	6a4265273f16420bb2661fbb	S001	26.55	50.89	74	2026-06-29 14:29:27	2026-06-29 14:29:27	72	v1.2.0
8	6a4265253f16420bb2661fba	S002	24.34	45.82	134	2026-06-29 14:29:25	2026-06-29 14:29:25	49	v1.0.3
9	6a4265233f16420bb2661fb9	S003	30.55	50.86	140	2026-06-29 14:29:23	2026-06-29 14:29:23	70	v2.0.1

Firmware versions reported (device metadata):

Figure 10: Dashboard view of raw telemetry documents stored in MongoDB.

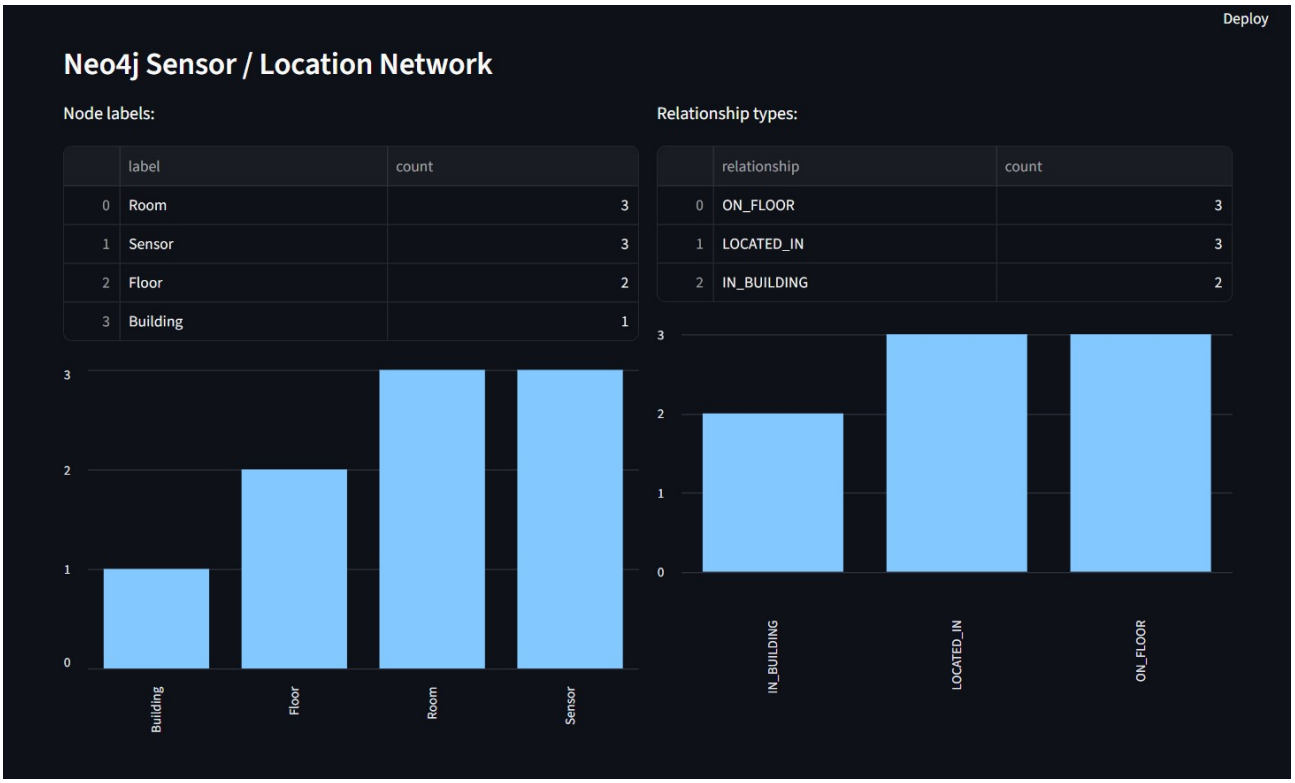


Figure 11: Dashboard summary of Neo4j node labels and relationship types.

9. Testing and Evaluation

Testing was done from the bottom of the system to the top. First, Docker services were started. Then the Python dependencies were installed, the subscriber was started, the publisher sent messages, and each database was checked directly. Finally, the dashboard was opened to confirm that the stored data could be visualized.

9.1 Environment and Dependency Tests

The project folder contains the publisher, subscriber, dashboard, database files, Docker Compose file, requirements files, and screenshots. This structure makes it easy to run and explain the project.

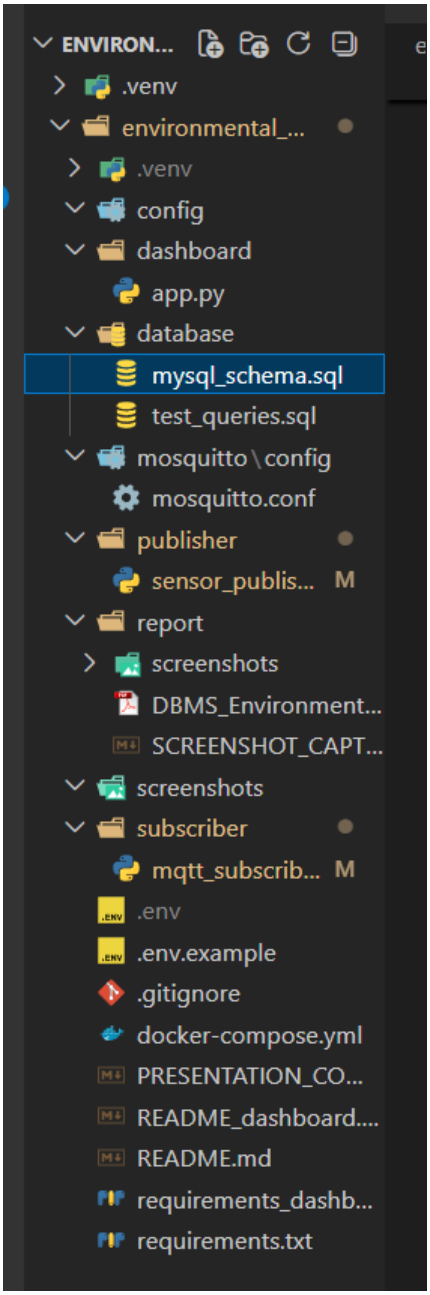


Figure 12: Project folder structure containing Docker, database, publisher, subscriber, dashboard, and report resources.

The Python environment was prepared with a virtual environment and the required libraries. The core pipeline uses paho-mqtt, mysql-connector-python, pymongo, neo4j, and python-dotenv. The dashboard uses Streamlit, pandas, and the same database drivers.

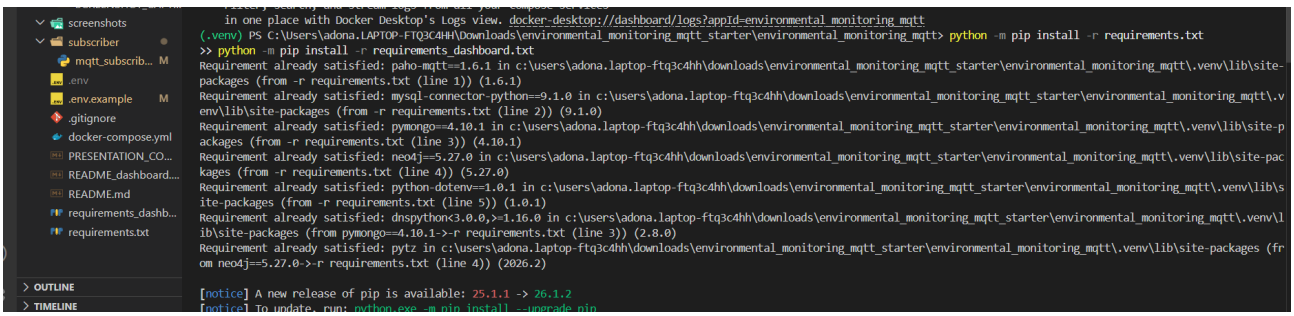


Figure 13: Installation and verification of main Python dependencies.

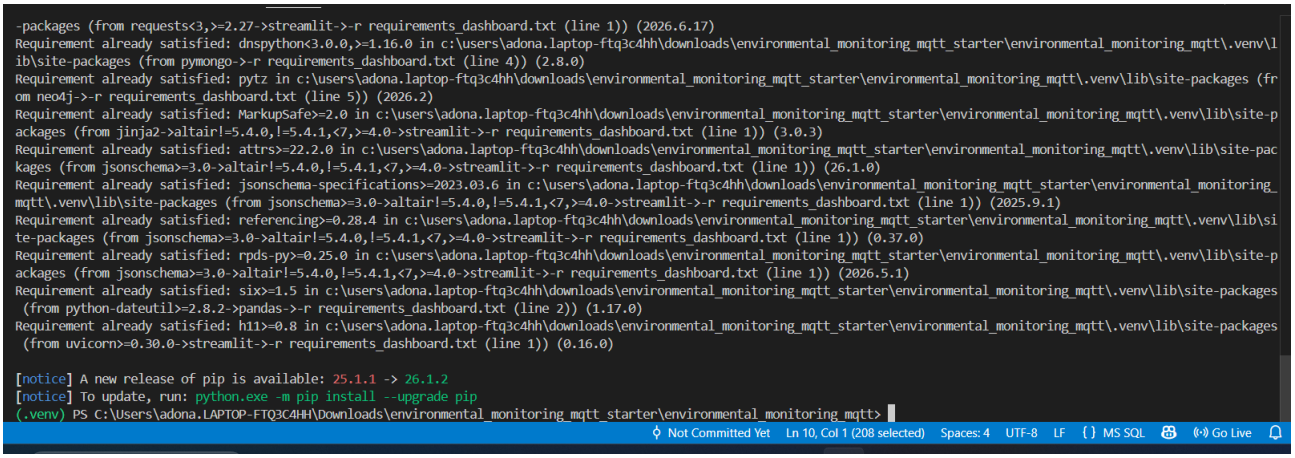


Figure 14: Installation and verification of dashboard dependencies.

9.2 MQTT Functional Test

The MQTT test confirmed that the publisher and subscriber communicate through the broker. The subscriber was started first so it could listen before any messages were published. The publisher then sent network, reading, and telemetry messages to the three MQTT topics. The subscriber output shows that messages were received and stored in the appropriate databases.

Test item	Expected result	Observed result
Subscriber connection	Connect to broker and subscribe to three topics	Subscriber connected to localhost:1883 and subscribed to network, readings, and telemetry
Publisher output	Publish topology and repeated sensor data	Publisher sent topology to Neo4j and repeated readings/telemetry to MySQL and MongoDB
Routing logic	Write based on topic	Subscriber printed MySQL, MongoDB, and Neo4j write messages separately

9.3 MySQL Storage and Analysis Tests

MySQL was tested using direct SQL queries. The readings table was checked by selecting the latest readings. The status column shows that the subscriber classification logic was applied before storage.

```
mysql> SELECT * FROM readings ORDER BY reading_id DESC LIMIT 10;
+------------+-----------+-------------+----------+-------------+---------------------+---------+---------------------+
| reading_id | sensor_id | temperature | humidity | air_quality | reading_time        | status  | created_at          |
+------------+-----------+-------------+----------+-------------+---------------------+---------+---------------------+
|         67 | S001      |       22.98 |    47.72 |          61 | 2026-06-29 14:18:17 | NORMAL  | 2026-06-29 12:18:17 |
|         66 | S001      |       23.59 |    77.09 |         153 | 2026-06-29 14:18:15 | DANGER  | 2026-06-29 12:18:15 |
|         65 | S003      |       34.74 |    36.11 |         165 | 2026-06-29 14:18:13 | DANGER  | 2026-06-29 12:18:13 |
|         64 | S003      |       19.65 |    80.41 |         131 | 2026-06-29 14:18:11 | DANGER  | 2026-06-29 12:18:11 |
|         63 | S002      |       21.87 |    80.47 |          74 | 2026-06-29 14:18:09 | DANGER  | 2026-06-29 12:18:09 |
|         62 | S003      |       28.73 |    48.97 |         136 | 2026-06-29 14:18:07 | WARNING | 2026-06-29 12:18:07 |
|         61 | S001      |       34.27 |    60.75 |         138 | 2026-06-29 14:18:05 | WARNING | 2026-06-29 12:18:05 |
|         60 | S003      |       28.76 |    48.56 |         136 | 2026-06-29 14:18:03 | WARNING | 2026-06-29 12:18:03 |
|         59 | S003      |       36.31 |    64.87 |         134 | 2026-06-29 14:18:01 | DANGER  | 2026-06-29 12:18:01 |
|         58 | S003      |       35.95 |    40.18 |         100 | 2026-06-29 14:17:59 | DANGER  | 2026-06-29 12:17:59 |
+------------+-----------+-------------+----------+-------------+---------------------+---------+---------------------+
10 rows in set (0.00 sec)
```

Figure 15: MySQL query showing the latest readings stored by the MQTT subscriber.

The alerts table was tested by joining it with alert_types. This confirms that alerts are linked to readings and that the readable message is taken from the alert type definition rather than being repeated directly in every reading record.

```
mysql> SELECT a.alert_id, a.reading_id, a.sensor_id, a.alert_type,      t.default_message AS message, a.alert_time FROM alerts a JOIN alert_types t ON a.ale
rt_type = t.alert_type ORDER BY a.alert_id DESC LIMIT 10;
+----------+------------+-----------+--------------------+-----------------------------------------------------------+---------------------+
| alert_id | reading_id | sensor_id | alert_type         | message                                                   | alert_time          |
+----------+------------+-----------+--------------------+-----------------------------------------------------------+---------------------+
|       76 |         92 | S001      | POOR_AIR_QUALITY   | Air quality is poor and above the dangerous threshold.    | 2026-06-29 14:19:07 |
|       75 |         91 | S003      | ENVIRONMENT_WARNING | One or more environmental values are above the normal range. | 2026-06-29 14:19:05 |
|       74 |         89 | S003      | HIGH_TEMPERATURE   | Temperature is above the dangerous threshold.             | 2026-06-29 14:19:01 |
|       73 |         88 | S003      | ENVIRONMENT_WARNING | One or more environmental values are above the normal range. | 2026-06-29 14:18:59 |
|       72 |         87 | S002      | ENVIRONMENT_WARNING | One or more environmental values are above the normal range. | 2026-06-29 14:18:57 |
|       71 |         85 | S001      | ENVIRONMENT_WARNING | One or more environmental values are above the normal range. | 2026-06-29 14:18:53 |
|       70 |         84 | S002      | ENVIRONMENT_WARNING | One or more environmental values are above the normal range. | 2026-06-29 14:18:51 |
|       69 |         83 | S002      | HIGH_HUMIDITY      | Humidity is above the dangerous threshold.                | 2026-06-29 14:18:49 |
|       68 |         82 | S003      | POOR_AIR_QUALITY   | Air quality is poor and above the dangerous threshold.    | 2026-06-29 14:18:47 |
|       67 |         81 | S001      | ENVIRONMENT_WARNING | One or more environmental values are above the normal range. | 2026-06-29 14:18:45 |
+----------+------------+-----------+--------------------+-----------------------------------------------------------+---------------------+
10 rows in set (0.00 sec)
```

Figure 16: MySQL JOIN query showing alerts with their default messages from alert_types.

The project also tested aggregation by counting readings per sensor. This confirms that the stored relational data can be analyzed using normal SQL grouping.

```
mysql> SELECT sensor_id, COUNT(*) AS total_readings FROM readings GROUP BY sensor_id;
+-----------+----------------+
| sensor_id | total_readings |
+-----------+----------------+
| S001      |             36 |
| S003      |             30 |
| S002      |             42 |
+-----------+----------------+
3 rows in set (0.00 sec)
```

Figure 17: SQL aggregation query counting total readings per sensor.

9.4 MongoDB Storage Test

MongoDB was tested from the shell by opening the env_monitoring database and checking the sensor_messages collection. The inserted documents show the advantage of document storage because the device field can contain different metadata depending on the message.

```
(.venv) PS C:\Users\adona.LAPTOP-FTQ3C4HH\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt> docker exec -it env_mongodb_db mongo
sh -u root -p rootpass --authenticationDatabase admin
Current Mongosh Log ID: 6a4262f13cbe9d6c4d9df8a2
Connecting to:          mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.8.3
Using MongoDB:          7.0.37
Using Mongosh:          2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-06-29T12:13:55.141+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/pro
dnotes-filesystem
------

test>
```

Figure 18: MongoDB shell connected to the database container.

```
env_monitoring> show collections
sensor_messages
env_monitoring> db.sensor_messages.find().pretty()
[
  {
    _id: ObjectId('6a42620a3f16420bb2661e2b'),
    sensor_id: 'S001',
    temperature: 21.31,
    humidity: 71.95,
    air_quality: 175,
    timestamp: '2026-06-29 14:16:05',
    device: { battery_level: 57, firmware_version: 'v2.0.1' },
    stored_at: '2026-06-29 14:16:10'
  },
  {
    _id: ObjectId('6a42620a3f16420bb2661e2c'),
    sensor_id: 'S003',
    temperature: 23.26,
    humidity: 73.75,
    air_quality: 56,
    timestamp: '2026-06-29 14:16:07',
    device: {
      battery_level: 61,
      firmware_version: 'v1.2.0',
      signal_strength_dbm: -59
```

Figure 19: MongoDB documents stored in the sensor_messages collection with flexible device metadata.

9.5 Neo4j Graph Test

Neo4j was tested in the Neo4j Browser. The database information panel shows 9 nodes and 8 relationships in the sensor/location network. The final labels are Building, Floor, Room, and Sensor. The final relationship types are IN_BUILDING, ON_FLOOR, and LOCATED_IN.

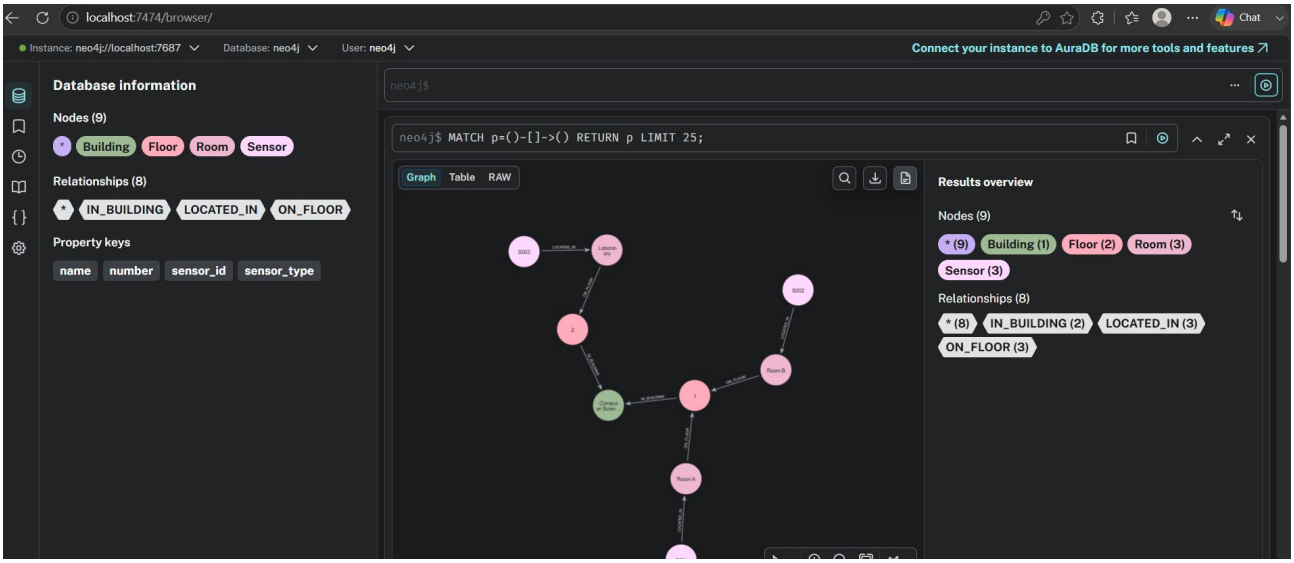


Figure 20: Neo4j Browser showing the complete sensor/location topology graph.

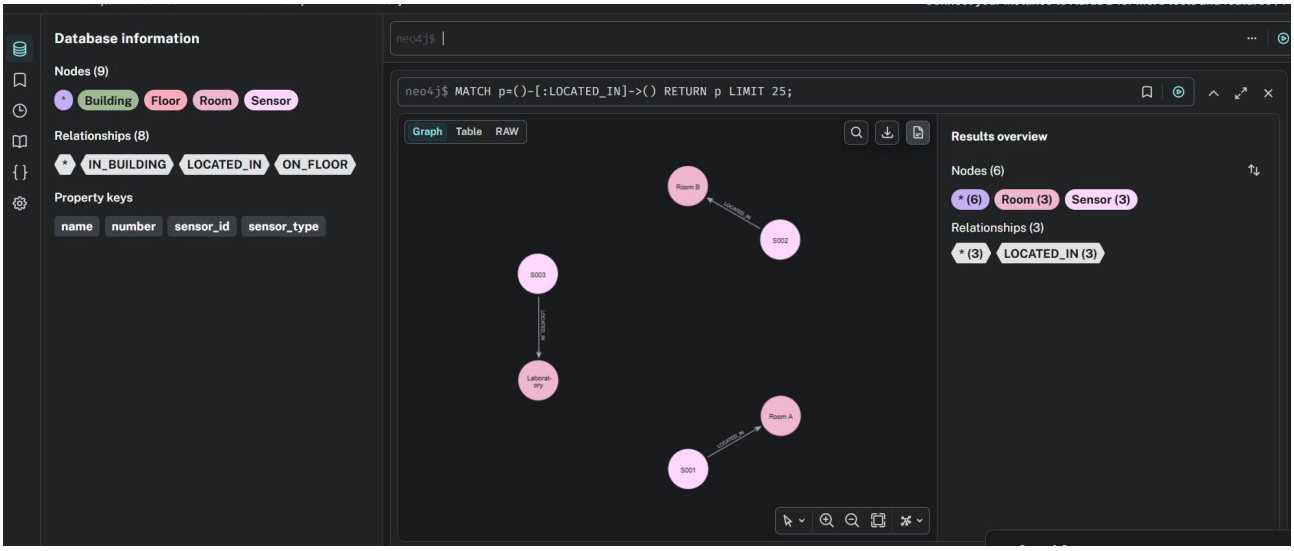


Figure 21: Neo4j query showing LOCATED_IN relationships between sensors and rooms.

9.6 Performance Evaluation

The subscriber records basic performance metrics while it runs. For each database write, it measures latency in milliseconds. At the end of execution, it prints a performance summary that includes the total number of stored messages, runtime, throughput in messages per second, average write latency per database, and error count.

$$\text{Throughput} = \frac{N}{\Delta t}$$

$$\overline{L}_{db} = \frac{1}{n_{db}} \sum_{k=1}^{n_{db}} \left(t_k^{end} - t_k^{start} \right)$$

The terminal output shows that the first Neo4j write can be slower because the driver and database connection are warming up. After that, Neo4j writes become much faster. MongoDB writes are usually very fast for individual telemetry documents, and MySQL writes include the extra work of classification, inserting the reading, and sometimes inserting an alert.

Metric	How it is measured	Purpose

Write latency	time.perf_counter() around each database write	Compare how long each database write operation takes
Throughput	total messages stored divided by runtime	Estimate processing speed
Reliability	error counter in the subscriber	Detect message-processing failures
Stored totals	count of writes per target database	Confirm that topic routing is working

9.7 Testing Summary

The tests confirmed that the final system works end to end. Docker starts the services, the publisher generates simulated IoT data, Mosquitto forwards the messages, the subscriber receives and processes them, and the data is stored in MySQL, MongoDB, and Neo4j according to topic. The dashboard confirms the result by reading and displaying all three database models.

10. Results Discussion

The final result is a working IoT data pipeline for environmental monitoring. The most important result is not only that data is stored, but that the storage is done according to the meaning of the MQTT topic. Structured readings are handled by MySQL, flexible telemetry is handled by MongoDB, and topology data is handled by Neo4j.

This gives the project a stronger database design than a system that simply copies the same message into every database. Each database has a clear responsibility. MySQL supports relational analysis, MongoDB preserves flexible device-level messages, and Neo4j represents the physical placement of sensors.

The dashboard results show hundreds of MySQL readings and MongoDB telemetry documents, while Neo4j contains a smaller and stable network graph. This difference is expected: readings and telemetry are continuous stream data, while the location graph changes only when the sensor topology changes.

11. Challenges Faced

The first challenge was coordinating several services at the same time. The project requires an MQTT broker, MySQL, MongoDB, Neo4j, and a Python application. Docker Compose solved this by making all services start with one command.

The second challenge was designing the data routing correctly. A simple version of the project could store the same message in all databases, but the project requirement is more meaningful when the MQTT topic decides the database. The final implementation solves this by using three topics and three different payload types.

The third challenge was connecting the dashboard to data stored in different models. The dashboard solves this by using sensor_id as a common logical link. MySQL stores the readings, while Neo4j stores the rooms for each sensor. The dashboard maps sensor_id to location to show readings with their physical room.

12. Conclusion

This project implemented an environmental monitoring system using MQTT, Python, MySQL, MongoDB, Neo4j, Docker, and Streamlit. The system simulates IoT sensors, sends data through a Mosquitto MQTT broker, processes messages with Python, and stores data in different databases according to message topic.

The project satisfies the main objective of integrating MQTT server communication with Python and multiple database platforms. It also demonstrates the practical value of choosing the right database model for each type

of IoT data. MySQL is used for structured readings and alerts, MongoDB is used for flexible telemetry, and Neo4j is used for the sensor/location graph.

The tests and screenshots confirm that the system works from message generation to dashboard visualization. Overall, the project successfully demonstrates topic-based IoT data ingestion, real-time processing, multi-database storage, and basic performance evaluation.

13. Future Improvements

The project can be improved in several ways. First, simulated sensors could be replaced with physical devices such as ESP32, Arduino, or Raspberry Pi sensors. Second, the system could support more sensors and more MQTT topics, such as energy consumption, smoke detection, or noise level monitoring.

Third, performance could be improved by using batch inserts, asynchronous writes, or a message queue between the subscriber and database writers. Fourth, the alert system could be extended to send email, SMS, or dashboard notifications when dangerous values are detected. Finally, the dashboard could include authentication and more advanced analysis, such as daily averages, anomaly detection, or predictions based on historical data.

14. Documentation and Presentation

The project can be reproduced using the following execution flow:

1. Start Docker services with docker compose up -d.
2. Install the pipeline dependencies with pip install -r requirements.txt.
3. Install the dashboard dependencies with pip install -r requirements_dashboard.txt.
4. Start the subscriber with python subscriber/mqtt_subscriber.py.
5. Start the publisher with python publisher/sensor_publisher.py.
6. Open the dashboard with streamlit run dashboard/app.py.

For presentation, the project should be explained as a topic-based data pipeline. The key point to emphasize is that the MQTT topic decides the storage destination. This is what connects the project directly to the requirement of storing sensor or IoT data in different database platforms based on message topic.

A short presentation flow can be: problem, architecture, MQTT topics, Python publisher and subscriber, MySQL design, MongoDB design, Neo4j graph, dashboard, testing, performance, and future improvements.

15. References

7. Docker Documentation. Docker Compose documentation. https://docs.docker.com/compose/
8. Eclipse Mosquitto Documentation. MQTT broker documentation. https://mosquitto.org/documentation/
9. Eclipse Paho Documentation. Python MQTT client documentation. https://www.eclipse.org/paho/
10. MySQL 8.0 Documentation. Relational database documentation. https://dev.mysql.com/doc/
11. MongoDB Documentation. Document database documentation. https://www.mongodb.com/docs/
12. Neo4j Documentation. Graph database and Cypher documentation. https://neo4j.com/docs/
13. Python Documentation. Python programming language documentation. https://docs.python.org/3/
14. Streamlit Documentation. Web application framework documentation. https://docs.streamlit.io/